

Executable Language Programs with Event-Aligned Spline Actions

Author names removed for blind review.

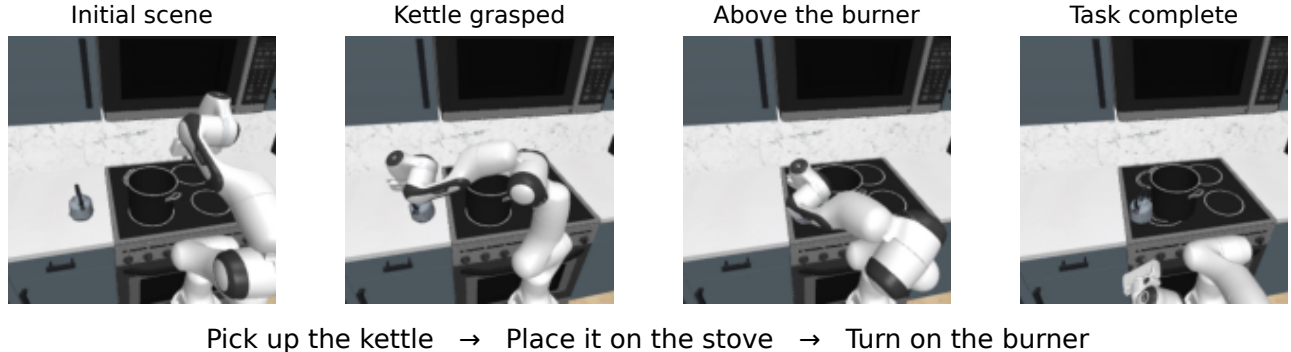


Fig. 1. A RoboCasa365 policy executes a kettle task through short language instructions. Each instruction conditions several event-aligned spline segments, each with its own predicted duration. The representation exposes two interfaces: which subgoal to execute and how to time its motion. Frames show one successful rollout; text below abbreviates the program, not the active instruction at each displayed frame.

Abstract—Long-horizon robot tasks contain distinct subgoals, but a vision-language-action policy is typically trained with one caption for an entire demonstration. We study an alternative interface: executable language clauses coupled to event-aligned spline actions. The policy predicts eight control points, a gripper curve, and a duration for each short motion segment, while successive clauses specify the current subgoal. We test whether this representation makes piecewise language execution more effective and whether its motion can be retimed after training. On two LIBERO-Long tasks, a matched three-seed study finds that clause-aligned training increases success under a shared clause schedule by 34.3 percentage points for waypoints and 43.7 points for the spline head, a 9.3-point interaction. Clause-order interventions change the first completed subgoal, and official phase labels improve two-task RoboCasa365 success on both evaluated seeds. Separately, retiming the same spline checkpoint improves mean CALVIN chain length from 1.31 to 1.69 across three seeds, with $1.42\times$ realized speed. These results support a compact action interface for piecewise language execution and explicit timing control, without changing the vision-language backbone.

I. INTRODUCTION

Consider the instruction *put the soup and tomato in the basket*. The robot must finish one object transfer, leave the placed object alone, and move to the next. A whole-task caption states the final goal but does not identify which subgoal is active. Vision-language-action (VLA) policies can infer that progress from images and robot state [1], [2], [3]; an alternative is to provide the instruction as a sequence of executable clauses. The question is then how to represent the motion generated under each clause.

We study *event-aligned spline actions*. Rather than a fixed array of future waypoints, each policy query predicts a short cumulative command trajectory and its duration. Eight cubic B-spline control points describe the pose-command path; a separate curve describes the gripper command. Training

segments end at a gripper transition, a sustained pause, or a duration cap. The active language clause conditions this same policy repeatedly until a scheduler advances to the next clause (Fig. 1). A clause is a semantic subgoal, not a single eight-token trajectory: several motion segments may be needed to complete it.

We test two properties of this interface. First, does the structured head make piecewise language supervision more effective? We compare spline and waypoint heads using identical physical demonstrations, matched fine-tuning budgets, and the same clause schedule at inference. Second, how well does the predicted motion execute under a change of clock? Both spline and waypoint paths can be resampled; the comparison should hold the controller and policy-query cadence fixed.

Our primary language experiment crosses action head, training labels, and inference interface. On two LIBERO-Long tasks, clause-trained spline policies reach 55.3% success under scheduled clauses, compared with 41.3% for clause-trained waypoint policies. Relative to whole-caption training under that *same schedule*, the spline head gains an additional 9.3 percentage points, positive in each of three seeds. The full factorial measures the trade-off between scheduled and static instructions. Reversing clauses provides a behavioral check, and official semantic phases reproduce the aggregate training-label benefit on longer RoboCasa365 programs.

On the official CALVIN chain benchmark, uniform decode-time retiming improves all three spline seeds and raises mean completed tasks from 1.31 to 1.69. This changes the timing of an already predicted motion, without retraining the policy. We distinguish this within-checkpoint benefit from the comparison against retimed waypoint outputs.

The paper makes two contributions:

- An event-aligned spline VLA interface and a controlled study of executable language, including a replicated head-label interaction, order interventions, and RoboCasa phase supervision.
- A shape-duration decoder that exposes the physical clock of each predicted segment, evaluated through fixed-checkpoint retiming and an explicit waypoint-interpolation comparison protocol.

II. RELATED WORK

Structured action representations. ACT and Diffusion Policy predict action chunks to model temporally extended behavior [4], [5]. VLAs combine such action prediction with pretrained visual and language representations [2], [3]; FAST compresses action sequences with frequency-domain tokens [6]. Several recent methods use splines. BEAST encodes action sequences as B-spline tokens [7]. B-spline Policy predicts continuous curves and accelerates execution by temporal rescaling [8]. Spline Policy studies continuous outputs across policy families and their use in resampling, editing, and downstream control [9]. Our contribution concerns event-duration supervision and the interaction between a structured action head and executable language. Continuous trajectories and temporal resampling are shared with this prior work.

Language and long-horizon execution. Large VLAs combine heterogeneous training data and semantic prediction to execute extended tasks [10]. Long-VLA uses phase-aware inputs [11], while tool-aligned VLAs separate high-level reasoning from policy invocation [12]. We instead study a single policy conditioned on successive clauses, with the physical training trajectories held fixed. Counterfactual instruction studies expose cases where visual or positional shortcuts dominate language [13], [14]. They motivate testing the requested behavior directly. Our order intervention therefore measures the first completed subgoal separately from final task success; changing the reward under an incompatible prompt alone would not establish successful steering.

Execution timing. PACE selects action-chunk execution horizons from low-speed transitions at inference [15]. Inference-time policy steering uses human interaction to guide generated behavior [16]. Here, demonstration events supervise segment duration, and a decoder controls its execution time. The clause scheduler and motion clock act at different scales: one chooses the active subgoal; the other determines how a predicted segment is sampled.

III. EVENT-ALIGNED SPLINE ACTIONS

Let o_t denote images, s_t robot state, and ℓ_t the active instruction. A conventional chunking VLA predicts H future actions from (o_t, s_t, ℓ_t) . We retain its vision-language backbone and flow-matching action expert, but change the prediction targets and decoder (Fig. 2).

A. A Short Motion Program

For a window of T pose-delta commands $a_0^{\text{pose}}, \dots, a_{T-1}^{\text{pose}}$, form the cumulative command path

$$p_0 = 0, \quad p_k = \sum_{j=0}^{k-1} a_j^{\text{pose}}. \quad (1)$$

This path is expressed in the controller’s command coordinates. It is not a reconstruction of the measured end-effector pose, and summing rotational command components is not an exact composition on $SO(3)$. We approximate it with a clamped, uniform cubic B-spline,

$$p(u) = \sum_{i=0}^{n-1} B_{i,3}(u)c_i, \quad u \in [0, 1], \quad n = 8. \quad (2)$$

The training fit pins $c_0 = 0$ and $c_{n-1} = p_T$. The remaining controls minimize

$$\sum_{k=0}^T \|p(k/T) - p_k\|_2^2. \quad (3)$$

We precompute the least-squares operators for each allowed duration. The gripper command is fitted directly, without cumulative summation, and thresholded by sign during decoding. In RoboCasa’s mobile-manipulator action layout, base and mode commands occupy additional fitted channels rather than pose-delta channels.

Each output token contains one pose control point, one gripper control value, and a replicated duration target $\log T$. The action expert therefore emits eight tokens rather than the waypoint head’s 50. We standardize targets per token and train with the same conditional flow-matching objective [3], [17]. At inference, we average the predicted log durations, $\bar{z} = \frac{1}{n} \sum_i \widehat{\log T}_i$, and decode

$$\hat{T} = \text{clip}(\text{round}(e^{\bar{z}}), T_{\min}, T_{\max}). \quad (4)$$

We reset the predicted starting control point to zero. The predicted endpoint remains learned; endpoint-preserving resampling preserves that prediction, not necessarily the demonstrated endpoint or the robot’s realized path.

B. Event-Aligned Supervision

Starting at each training frame, we select the first boundary after $T_{\min} = 8$ commands. A boundary is a gripper-command transition, two consecutive pose-command norms below 0.15 times the window median, or episode padding. If none occurs, the window ends at $T_{\max} = 24$ commands (16 for CALVIN). A boundary at index k gives $T = k$: commands $0, \dots, k-1$ enter the fit, while the boundary command belongs to the next segment. Thus duration predicts an action event or cap, not a semantic subgoal label.

This distinction matters for language supervision. Gripper and pause events provide short motion boundaries, but a task clause may require several grasps, transports, or contact corrections. Event-aligned targets do not impose a one-to-one correspondence between language clauses and spline segments.

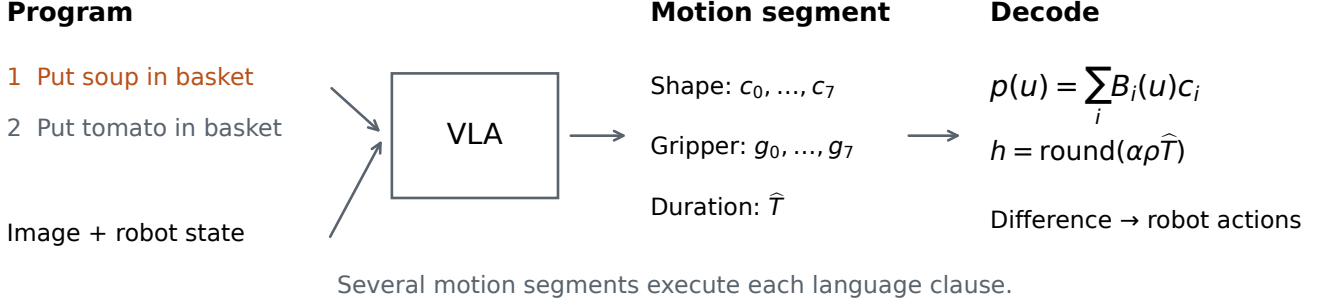


Fig. 2. The active clause conditions an unchanged VLA backbone. Each query produces a short spline segment and its duration. The decoder chooses the number of issued actions using duration scale α and control-rate ratio ρ . Clause switching occurs on a separate, slower schedule; it is held identical across heads in the primary language comparison.

C. Executable Clauses

A program is an ordered list $\mathcal{L} = (\ell^1, \dots, \ell^M)$. At any time, the active clause replaces the whole-task caption in the policy input. A scheduler advances the clause pointer and clears queued actions generated under the previous instruction. There is no additional high-level model.

For the two-subgoal LIBERO study, we label each demonstration with two clauses. The split is the first gripper release following at least 48 consecutive closed commands; the release is assigned to the first clause, and the next observation–action pair begins the second. The two training views have identical images, states, actions, and episode membership. Only language metadata changes.

The main comparison uses an outcome-independent, fixed clause clock for both heads. The switch occurs at the median labeled transition in the training demonstrations: step 139 for the soup/tomato task and 107 for the two-mug task. A second condition switches at an executed close-to-open gripper command. The order diagnostic instead uses the official subgoal predicate to advance the clause; this privileged scheduler isolates the order request from completion-detection errors and is not a deployable result.

D. Controlling the Physical Clock

Let f_0 be the action rate used to interpret the training commands, f the execution rate, and $\rho = f/f_0$. A duration multiplier α yields

$$\begin{aligned} h &= \max\{2, \text{round}(\alpha \rho \hat{T})\}, \\ a_j^{\text{pose}} &= p((j+1)/h) - p(j/h). \end{aligned} \quad (5)$$

Ignoring rounding and actuator clipping, execution time is $h/f \approx \alpha \hat{T}/f_0$. Changing α therefore changes pace; changing ρ at fixed α changes sampling density while preserving duration. The underlying predicted curve is unchanged. The gripper curve is sampled on the corresponding grid and sign-thresholded. As with the original controller, issued pose commands are bounded.

This factorization guarantees a shared predicted endpoint across sampling grids before clipping. It does not guarantee unchanged closed-loop outcomes, tracking at arbitrary speeds, or derivative continuity across replans. We use ordinary receding-horizon execution: issue up to K actions from

a segment, observe the robot again, and replan. A segment shorter than K drains the queue earlier.

Waypoint interpolation baseline. The same timing operation can be applied to a waypoint prediction. Denormalize its per-step deltas, accumulate them into a path q_k , linearly interpolate that path at $h+1$ equally spaced phases, and difference successive samples. The gripper uses nearest-step sampling and sign thresholding. Finally, renormalize if the policy’s saved postprocessor expects normalized actions. Interpolating normalized deltas directly is incorrect: adding a per-step mean after changing the number of commands changes the total displacement. This baseline needs no re-training and is distinct from repeating unchanged waypoints at another rate.

IV. EXPERIMENTS

We first test executable language and then physical timing, with broad benchmark success providing context for both interventions.

A. Setup

Policies and data. All heads use the same 450M-parameter SmolVLA backbone [3]. Waypoint A predicts 50 action tokens; fixed-window spline B uses eight control points and a 20-command target; event spline C uses eight control points and predicts duration. The broad LIBERO study trains each for 100,000 updates with batch size 32 on the same 40-task dataset, with three training seeds. The clause study fine-tunes A and C from head-matched 100k checkpoints, crossing whole-task and clause-aligned training labels. It uses 7,500 updates, batch size 32, and the same frozen physical episodes for every treatment. Evaluation uses the final checkpoint, not a best-of-evaluation selection.

Evaluation. LIBERO [18] supplies four ten-task suites. The language experiment uses two multi-object LIBERO-Long tasks: soup and tomato into a basket (t0), and two mugs onto different plates (t4). Each condition evaluates the same 50 simulator states per task and three training seeds. The primary comparison sets $K = 10$ and the same fixed clause schedule for both heads. Exact replays verify initial observations, simulator states, prompts, action traces,

TABLE I

LIBERO-LONG T0/T4 SUCCESS (%). EACH CELL CONTAINS 300 ROLLOUTS: 50 STATES/TASK $\times$ TWO TASKS $\times$ THREE TRAINING SEEDS. COLUMNS CHANGE INFERENCE; ROWS CHANGE TRAINING LABELS.

Head	Training labels	Inference instruction	
		Static whole	Clauses
Waypoint	Whole-task	49.0	7.0
Waypoint	Clauses	39.7	41.3
Event spline	Whole-task	57.0	11.7
Event spline	Clauses	33.0	55.3

and outcomes. Replay repeats are checks, not additional independent trials.

RoboCasa365 [19] tests KettleBoiling and RinseSinkBasin on a targeted scene split, with official full-task success predicates. CALVIN [20] uses the official list of five-instruction chains and reports mean consecutive tasks completed. The existing timing study evaluates 1,000 chains per checkpoint seed. Success percentages always refer to complete tasks unless a stage metric is named explicitly.

Uncertainty. For the primary language interaction, we report the three checkpoint-seed effects and the t interval over those seeds. A hierarchical bootstrap resamples seeds and paired task/state outcomes as a sensitivity analysis. For binary paired interventions we use exact McNemar tests; RoboCasa’s reported phase-label tests use Fisher’s exact test. CALVIN’s historical intervals are paired-chain intervals for the evaluated checkpoints, not population-level intervals over possible training runs.

B. Piecewise Language Execution

The language dataset contains 33 demonstrations of t0 and 38 of t4: 71 episodes and 19,378 frames. Table I crosses the training labels with the inference interface. The scheduled columns use the same clauses and transition times regardless of training treatment. This holds the requested language program fixed while testing whether the policy learned to execute it.

Under scheduled clauses, success rises from 7.0% to 41.3% for the waypoint head and from 11.7% to 55.3% for the event spline. The label effects are +31, +30, and +42 points for A, and +41, +42, and +48 for C (Fig. 3). Define the head interaction at the fixed schedule as

$$\Delta = (S_{C,\text{clause}} - S_{C,\text{whole}}) - (S_{A,\text{clause}} - S_{A,\text{whole}}), \quad (6)$$

where the second subscript denotes *training labels*. Its mean is +9.33 points, with a training-seed 95% t interval of [1.74, 16.92] and hierarchical interval [0.67, 18.0]. The advantage is therefore not limited to a single training run. It concerns the event-spline head as a whole; this experiment does not separately identify the contributions of control-point count, event boundaries, and duration supervision.

The static columns provide an important control. Whole-caption-trained policies remain competent when given their native whole-caption input. Clause training does not improve

TABLE II

ORDER INTERVENTION. FIRST-OBJECT FLIPS ARE MEASURED ON T4 (50 PAIRED STATES); FINAL TASK COUNTS POOL T0/T4 (100 STATES). ALL POLICIES HERE ARE CLAUSE-TRAINED.

Head	First-object flips		Final success	
	Requested	Opposite	Normal	Reverse
Waypoint	10/50	0/50	49/100	16/100
Event spline	10/50	0/50	55/100	21/100

this interface: it reduces static-prompt success by 9.3 points for A and 24.0 for C. The useful result is improved *addressability through a clause program*, not a universal increase in canonical-task success. With the spline head, programmed success reaches 55.3%, close descriptively to the whole-caption model’s 57.0%, while allowing the subgoal instruction to change during execution.

Switching without a fixed demonstration clock. Advancing the spline’s clauses on its executed gripper-release command yields 164/300 successes for clause training and 43/300 for whole-caption training: a 40.3-point label effect. This retains the direction and scale of the fixed-clock result. It uses an action-derived switch, not the scalar duration prediction or an oracle goal predicate.

C. Changing the Requested Subgoal Order

We reverse the two clauses and score which official goal predicate becomes true first. This diagnostic uses a privileged predicate to advance to the second clause. On t4, both heads exhibit 10/50 paired states in which the first completed object changes in the requested direction, with no changes in the opposite direction (two-sided sign test $p = .00195$). Of 12 states with a unique first goal under both orders, 10 switch as requested.

The intervention changes behavior, but final conjunction success falls under reversed order (Table II). The policy can respond to the first requested subgoal without reliably preserving it while completing the second. This separates first-subgoal steering from compositional generalization, and does not imply a spline-specific order advantage.

D. Semantic Phases on RoboCasa365

RoboCasa tests whether language alignment helps longer sequences involving grasping, transport, and fixture actuation. We use the same event-spline architecture and relabel identical demonstrations with official semantic phases. KettleBoiling has 501 demonstrations and 228,349 frames; RinseSinkBasin has 509 demonstrations and 211,036 frames. Within each seed, whole-caption and phase-label policies share initialization and receive 30,000 updates at batch size 32. Both are evaluated under the same outcome-independent phase schedule: Kettle switches at steps 128 and 242; Rinse switches at 162. These are demonstration-derived constants, not test-time success signals.

The two-task aggregate improves by 12 points on seed 1000 and 15 on seed 1001 (Table III). The first aggregate has Fisher $p = .00808$. Rinse improves on both seeds; Kettle’s

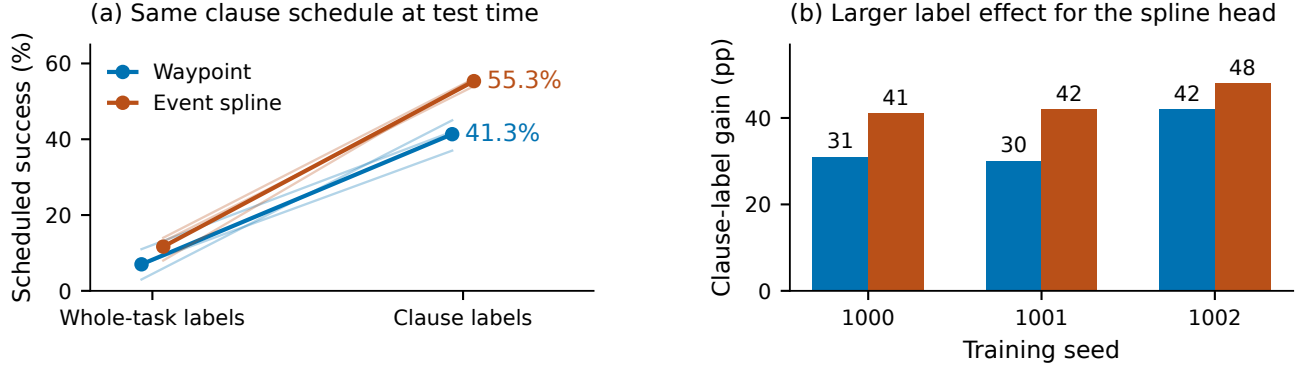


Fig. 3. Clause-aligned training improves execution under the same fixed clause schedule. (a) Absolute success on t0/t4; faint lines are individual training seeds and thick lines their means. (b) Within-head training-label effects for every seed. The spline-minus-waypoint interaction is +10, +12, and +6 percentage points.

TABLE III

ROBOCASA FULL-TASK SUCCESS COUNTS UNDER THE SAME FIXED PHASE SCHEDULE. n GIVES THE NUMBER OF TRIALS PER ROW AND LABEL TREATMENT. COMBINED ROWS SUM THE TWO TASKS.

Seed	Task	n	Whole labels	Phase labels
1000	Kettle	50	2	6
	Rinse	50	2	10
	Combined	100	4	16
1001	Kettle	20	2	1
	Rinse	20	1	8
	Combined	40	3	9

improvement does not replicate in the smaller second-seed evaluation. The result is a two-task, target-scene label comparison, not an estimate of performance over all 365 tasks or a waypoint-spline interaction on RoboCasa.

Stage measurements help locate the improvement. On seed 1000, phase labels increase Kettle grasp attainment from 24/50 to 39/50, and Rinse attainment of water-on with at least one washed region from 7/50 to 22/50. The latter improves from 3/20 to 12/20 on seed 1001. These intermediate counts are not substituted for full-task success.

The teaser follows one successful phase-trained Kettle rollout from a paired evaluation. Its whole-caption-trained counterpart begins from the identical observation and also grasps and reaches the stove. Only the phase-trained policy completes burner actuation: success at 516 steps versus failure at the 1,500-step limit. This paired example localizes the difference to the final stage.

E. A Controllable Physical Clock

The timing experiments change the decoder of an already trained policy. CALVIN is a useful test because its human demonstrations include pauses and variable pace. The event head uses eight control points, $T_{\min} = 8$, and $T_{\max} = 16$ at the policy’s 10-Hz action rate. Each command is held for three native simulator steps, preserving the replay-validated

TABLE IV

CALVIN MEAN CONSECUTIVE TASKS COMPLETED. RETIMING USES $\alpha = 0.6$, WITH ACTUATOR CLIPPING AND NO FEASIBILITY STRETCH. EACH SEED HAS 1,000 CHAINS.

Policy / decoder	1000	1001	1002	Mean
Waypoint / native	1.510	1.764	1.416	1.563
Spline / native	1.422	1.239	1.273	1.311
Spline / retimed	1.734	1.579	1.759	1.691

action convention. Every subtask receives the official 360-simulator-step budget.

Uniform retiming improves the spline in every seed (Table IV), with a pooled paired-chain gain of 0.379 completed tasks (95% CI [0.318, 0.441]). Realized speed is $1.42 \times$ the unretimed spline, rather than the nominal $1/0.6 = 1.67 \times$: rounding, replanning, and changed behavior affect completion time. The retimed spline exceeds the native waypoint by 0.127 tasks on average (paired-chain CI [0.063, 0.191]), while first-task success is similar, 69.9% versus 69.4%. Figure 4 shows that the difference is concentrated in sustaining longer chains. The waypoint comparison varies by training seed; the within-spline retiming improvement does not.

Separating interpolation from learned representation.

Native waypoint replay does not answer whether a spline is a better representation for retiming. We therefore specify a four-arm comparison: native and linearly retimed waypoints, and native and uniformly retimed event splines. Both retimed arms use $\alpha = 0.6$, raw-command interpolation/decoding, the same bounds, and no feasibility stretch. All four issue five policy-rate actions per neural query, so even the shortest retimed spline segment fits the common cadence. They use the same official chains, per-chain/subtask noise seeds, controller rate, and time budgets. Identity, nonzero-mean normalization, and endpoint tests precede closed-loop evaluation.

Revision-stage comparison: quantitative results from this matched-cadence experiment will replace this paragraph

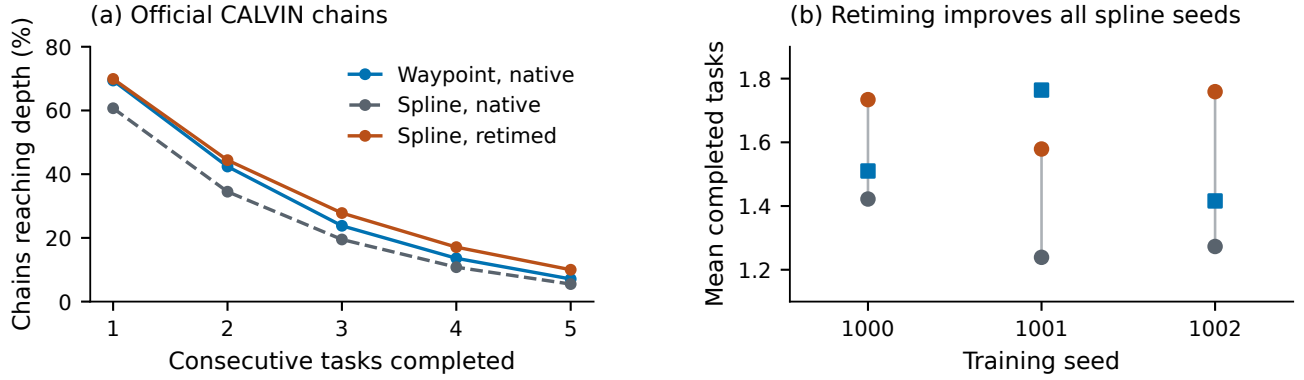


Fig. 4. CALVIN timing study, three checkpoint seeds and 1,000 official chains per seed. (a) Fraction reaching each chain depth. (b) Individual checkpoint results; vertical lines connect each spline before and after retiming. The waypoint comparator here uses its native decoder, not retimed interpolation.

TABLE V

BROAD LIBERO SUCCESS (%). THREE SEEDS, 1,200 ROLLOUTS/HEAD ACROSS FOUR SUITES. THE FINAL COLUMN IS MEAN $\pm$ SEED STANDARD DEVIATION OF SUITE-AVERAGED SUCCESS.

Head	Object	Spatial	Goal	Long	Average
Waypoint	94.7	84.3	86.0	63.3	82.1 $\pm$ 1.8
Fixed spline	96.3	77.7	84.0	65.3	80.8 $\pm$ 2.5
Event spline	92.3	77.3	89.3	63.7	80.7 $\pm$ 1.3

after the complete frozen evaluation. The historical results above establish useful spline retiming, not superiority to an interpolated waypoint baseline.

F. General Task Performance

Table V gives the four-suite context for these focused studies. Across three seeds, the event head averages 80.7% against 82.1% for waypoints, a 1.4-point difference at the native rate. The two-task language factorial is evaluated separately and is not folded into these suite averages.

G. Real-Robot Evaluation

Reserved for hardware results; no physical trials are claimed in this draft. The planned setup uses one 7-DoF arm with a linear base, a gripper, and handheld teleoperation. The base remains fixed unless reach requires it. Existing printed fixtures support a short drawer program: open, insert an object, then close or leave open. The two final clauses share the same prefix, making the requested behavioral difference directly visible. A second feasible object-ordering task can test a changed sequence if demonstration collection permits.

The final section will report the collected demonstrations, train/test program split, matched policy comparisons, and complete-task counts on frozen initial configurations. A paired late-clause filmstrip and prefix-versus-late trajectory comparison will test whether the policy changes behavior at the requested stage. Allocate approximately one page, including the result table and images. This is a focused test of language execution, not simply a deployment photograph.

V. DISCUSSION AND CONCLUSION

Event-spline policies benefit more from clause-aligned training than waypoint policies under an identical clause schedule. This interaction repeats across three seeds. The complete interface factorial shows the resulting trade-off: adapting to scheduled clauses improves their execution but reduces performance with a static whole-task caption. Order interventions show a corresponding change in the first completed subgoal, while RoboCasa extends the label comparison to longer manipulation sequences.

Several boundaries remain. The head comparison uses one VLA backbone and two LIBERO tasks, and combines event alignment, compactness, and duration prediction. Reversed-order completion remains difficult, so the current results do not establish general recombination of learned skills. Fixed clause clocks also assume predictable phase timing; the action-release condition relaxes that assumption only for tasks with a suitable gripper event.

Separating a predicted path from its duration lets the executor retime a motion without retraining the policy. CALVIN demonstrates a within-checkpoint improvement in both completion and pace; the matched waypoint comparison tests how much of this benefit depends on the representation. Together, the two studies evaluate spline actions as an interface for specifying subgoals and timing motion.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, *et al.*, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [3] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [4] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems*, 2023.

- [5] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems*, 2023.
- [6] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, "FAST: Efficient action tokenization for vision-language-action models," *arXiv preprint arXiv:2501.09747*, 2025.
- [7] H. Zhou, W. Liao, X. Huang, Y. Tang, F. Otto, X. Jia, X. Jiang, S. Hilber, G. Li, Q. Wang, *et al.*, "BEAST: Efficient tokenization of B-splines encoded action sequences for imitation learning," in *Advances in Neural Information Processing Systems*, 2025.
- [8] X. Han, H. Xiong, H. Chen, C. Liu, A. Torralba, Y. Zhu, and Y. Du, "B-spline policy: Accelerating manipulation policies via B-spline action representations," *arXiv preprint arXiv:2607.09648*, 2026.
- [9] M. Tian, Y. Li, S. Liu, A. Ijspeert, and S. Calinon, "Spline policy: A structured representation for robot policies," *arXiv preprint arXiv:2606.07386*, 2026.
- [10] Physical Intelligence, K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, *et al.*, " $\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.
- [11] Y. Fan, P. Ding, S. Bai, X. Tong, Y. Zhu, H. Lu, F. Dai, W. Zhao, Y. Liu, *et al.*, "Long-VLA: Unleashing long-horizon capability of vision language action model for robot manipulation," in *Conference on Robot Learning*, 2025.
- [12] Z. Lei, C. Liu, Y. Xiong, M. Xiong, Y. Ding, Z. Zhang, W. Li, and S. Chen, "Towards long-horizon embodied agents with tool-aligned vision-language-action models," *arXiv preprint arXiv:2605.13119*, 2026.
- [13] C. Glossop, W. Chen, A. Bhorkar, D. Shah, and S. Levine, "CAST: Counterfactual labels improve instruction following in vision-language-action models," *arXiv preprint arXiv:2508.13446*, 2025.
- [14] Y. Fang, Y. Feng, D. Jing, J. Liu, Y. Yang, Z. Wei, D. Szafir, and M. Ding, "When vision overrides language: Evaluating and mitigating counterfactual failures in VLAs," *arXiv preprint arXiv:2602.17659*, 2026.
- [15] J. Nie, J. Li, J. Zhang, J. Lao, C. Liu, T. Zhang, L. Lin, and S. Huang, "PACE: Phase-aware chunk execution for robot policies with action chunking," *arXiv preprint arXiv:2606.00537*, 2026.
- [16] Y. Wang, L. Wang, Y. Du, B. Sundaralingam, X. Yang, Y.-W. Chao, C. Perez-D'Arpino, D. Fox, and J. Shah, "Inference-time policy steering through human interactions," in *IEEE International Conference on Robotics and Automation*, 2025.
- [17] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations*, 2023.
- [18] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, 2023.
- [19] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu, "RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots," in *International Conference on Learning Representations*, 2026.
- [20] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, "CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks," *IEEE Robotics and Automation Letters*, 2022.